

- Date of entry:

12-5-2026

- What I've worked on:

I started by loading the spinal cord image. I used color filtering to select the blue/purple area. This area was important because I wanted to place the simulated neurons inside this part of the image.

- What problems I encountered:



At first, my HSV mask was too strict so way to many pixels that did contain neural tissue did not get seen as neural tissue, hence the few white dots at the picture above. In order to fix this, I simply lowered the mask. There were a few blue points near the edge of the picture, but I decided to leave them in due to time constraints.

- What I learned:

I learned about masking and that it is important not to overestimate of the pixels. More important, I learned how to approach a problem I have no previous experience in, I have never used a similar technique and still got a decent result.

- Which resources did I use:

https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_colorspaces/py_colorspaces.html

(I used alternative sources as well but this is the most important one)

- Date of entry:

14-5-2026

- What I've worked on:

I created 500 simulated neurons. Each neuron got a position, a threshold and a firing status. I also made a simple function to calculate the field strength around the electrode.

- What problems I encountered:

At first, I thought of making a .self-like function of the neurons. However, I quickly realised that it would be better to use pandas dataframes because of the large number of neurons I wanted simulate.

- What I learned:

Honestly, when it comes to the neurons themselves I did not learn a lot of new or interesting techniques. I did learn a bit more about pandas dataframes and how to store a large number of variables.

- Which resources did I use:

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

- Date of entry:

15-5-2026

- What I've worked on:

I loaded the muscle data from the CSV file. I cleaned the data and changed the coordinates so the muscle points fitted on the image. Then I calculated which muscle was closest to each neuron.

- What problems I encountered:

At first, the points from the research did not fit inside of the blue area at all. However, when adjusting the scale (the distance between the points). I could make them fit inside of the blue area. The literature of the research made me believe that the scientist mapped out the entirety of the blue area of the spinal cord, but as seen in my program, this was not the case. The points did form the "butterfly" I thought they were going to form, so I placed it in the most likely place for them to be in the spinal cord (one of the ventral horns).

- What I learned:

I learned coding a slider in which I could change the X, Y and scale of the points provided by the CSV-file (this program itself is not included in the final version of the code).

- Which resources did I use:

https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html>

- Date of entry:

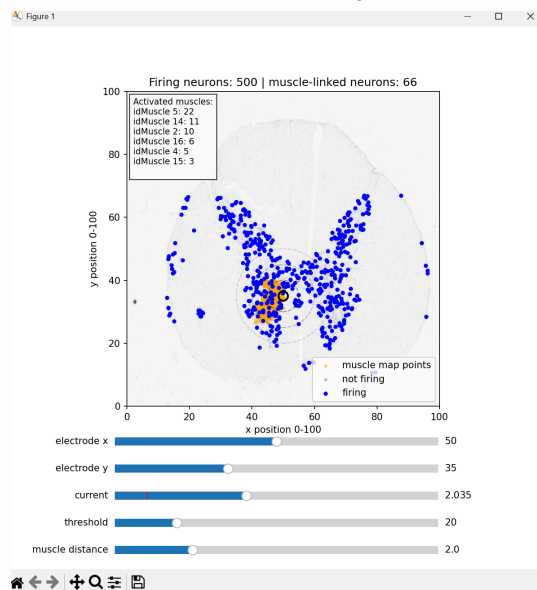
16-5-2026

- What I've worked on:

I made a plot with sliders. With the sliders, I could change the electrode position, current, threshold and muscle distance. The plot shows which neurons are firing and which neurons are connected to a muscle.

- What problems I encountered:

When making the slider, I overestimated the amount of electromagnetic field the neurons need to activate. Because of this mistake, when adjusting the slider, all the neurons will be activated when you are at roughly 1/3th of the slider, making the interface not user-friendly.



- What I learned:

I learned how to create an interface with sliders, which is something I have not done before. This is something I have not done before, and it took a few tries to get it right. Also I learned to think about user-friendliness when making a visualisation.

- Which resources did I use:

https://matplotlib.org/stable/gallery/widgets/slider_demo.html

https://matplotlib.org/stable/api/_as_gen/matplotlib.patches.Circle.html

- Date of entry:

17-5-2026

- What I've worked on:

I added a function that tests many different electrode positions and current values. The goal was to find the best setting for one target muscle. The program gives a higher score when the target muscle is activated and a lower score when other muscles are activated.

- What problems I encountered:

At first, I tried to make a live visual simulation of the optimization. However, this made the program too slow, because my computer had to update the plot while

testing many combinations. Because of this, I changed it to a faster optimization method without a full live animation.

- What I learned:

I learned how to test many possible stimulation settings with loops. I also learned how to give each setting a score, so the program can compare the results and choose the best electrode position and current. I learned that a faster method is sometimes better than a live visual simulation, because the computer does not have to update the plot after every test. This is also the reason the program names it fast optimizing.

- Which resources did I use:

<https://numpy.org/doc/stable/reference/generated/numpy.arange.html>

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sort_values.html

- Date of entry:

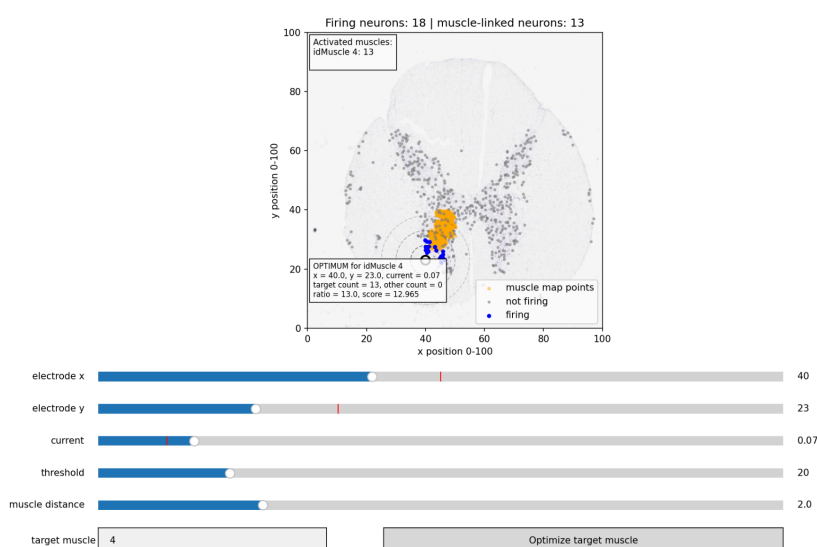
18-5-2026

- What I've worked on:

I added a text box where the user can choose the target muscle. I also added a button to start the optimization. The best electrode position is shown with a red star, and the result is shown in a pop-up window. The results are also saved as CSV files.

- What problems I encountered:

When adding the window in which the optimum would be displayed inside of the image instead of under it. In order to fix this, I decided to just make an entirely different image to display the results of the optimisation.



- What I learned:

I learned how to use a TextBox so the user can choose a target muscle. I also learned how to connect a Button to a function, so the optimization starts when the button is clicked. I learned that showing too much text inside the main plot can make the image unclear. Because of this, I made a separate pop-up window for the best result. I also learned how to save the optimization results as CSV files, so the results can be checked later.

- Which resources did I use:

<https://matplotlib.org/stable/gallery/widgets/textbox.html>

<https://matplotlib.org/stable/gallery/widgets/buttons.html>

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html

https://www.w3schools.com/python/python_try_except.asp